



Documentación del Proyecto Farmacia Inclusiva

Aplicación web administrativa para una farmacia, documentada desde su arquitectura, flujo de datos, backend, dependencias, validaciones, seguridad y estructura de carpetas.

Tipo de documento	Documentación técnica para presentación universitaria
Stack principal	React 19, Vite, Express, PostgreSQL 16, Docker Compose y Recharts
Proyecto	Aplicación frontend y backend en un mismo repositorio con un solo <code>package.json</code>
API base	<code>http://localhost:3001/api</code>

1. Resumen General Del Proyecto

Farmacia Inclusiva es una aplicación web administrativa diseñada para gestionar operaciones comunes de una farmacia: control de medicamentos, lotes, proveedores, clientes, usuarios y ventas. La aplicación permite consultar información, registrar nuevas operaciones y visualizar datos relevantes desde un panel organizado por módulos.

El proyecto está construido como una aplicación full stack en un solo repositorio. El frontend vive en `src/` y está desarrollado con React 19, Vite y `react-router-dom`. El backend vive en `backend/src/` y está desarrollado con Node.js y Express. La base de datos es PostgreSQL 16 y se levanta mediante Docker Compose usando el puerto local `5433`.

La relación entre las tres capas se puede resumir así: React muestra la interfaz y consume servicios en `src/services`; esos servicios hacen peticiones HTTP a la API Express; Express procesa la solicitud mediante rutas, controladores y modelos; finalmente, los modelos consultan o modifican la base de datos PostgreSQL.

Usuario en navegador

-> Componente React

-> Servicio frontend en `src/services`

-> API Express en `backend/src`

-> Modelo backend

-> PostgreSQL en Docker

-> Respuesta JSON

-> Estado de React

-> Renderizado en pantalla

Capa	Tecnología	Ubicación	Responsabilidad
Frontend	React 19 + Vite	<code>src/</code>	Interfaz, rutas, formularios, tablas, filtros, modales y gráficos.
Backend	Node.js + Express	<code>backend/src/</code>	API REST, validación de entrada, lógica de negocio y acceso a datos.
Base de datos	PostgreSQL 16	<code>BaseDeDatos/</code> y Docker	Persistencia de usuarios, clientes, medicamentos, lotes, ventas y catálogos.

2. Ubicación Del Backend

El backend se encuentra dentro de la carpeta `backend/src/`. No es un proyecto separado con su propio `package.json`; forma parte del mismo proyecto general y usa las dependencias declaradas en el `package.json` raíz.

Archivos principales

Ruta	Función
<code>backend/src/server.js</code>	Punto de arranque del backend. Carga variables de entorno, prepara la base de datos y levanta Express en el puerto configurado.
<code>backend/src/app.js</code>	Configura la aplicación Express: CORS, JSON, logging básico, rutas bajo <code>/api</code> y middleware de errores.
<code>backend/src/routes/index.js</code>	Agrupar las rutas principales: salud, auth, usuarios, proveedores, medicamentos, lotes, ventas y clientes.

Subcarpetas del backend

Carpeta	Contenido	Ejemplo
<code>routes/</code>	Define endpoints HTTP y conecta cada ruta con su controlador.	<code>ventasRoutes.js</code> define <code>GET /ventas</code> y <code>POST /ventas</code> .
<code>controllers/</code>	Recibe la petición, valida datos, llama al modelo y responde en JSON.	<code>ventasController.js</code> contiene <code>listarVentas</code> y <code>crearVenta</code> .
<code>models/</code>	Contiene las consultas SQL y operaciones directas contra PostgreSQL.	<code>ventasModel.js</code> consulta ventas, detalle y registra ventas en una transacción.
<code>validators/</code>	Valida datos recibidos antes de enviarlos al modelo.	<code>ventasValidator.js</code> valida usuario, método de pago y detalles de venta.
<code>middleware/</code>	Contiene funciones intermedias de Express.	<code>errorHandler.js</code> responde errores internos con estado 500.
<code>config/</code>	Configuración de entorno y base de datos.	<code>database.js</code> crea el <code>Pool</code> de PostgreSQL.
<code>utils/</code>	Utilidades reutilizables del backend.	<code>password.js</code> genera y verifica hashes de contraseña con PBKDF2.

Cómo se levanta

```
npm run db:up      # levanta PostgreSQL en Docker
npm run dev:api    # levanta el backend Express en http://localhost:3001
```

El servidor usa `PORT=3001` por defecto y monta las rutas bajo `/api`. Por ejemplo, el endpoint de salud queda disponible en `/api/salud`.

3. Cómo Se Muestran Los Datos En Pantalla

Los datos se muestran siguiendo un flujo por capas. El backend obtiene información desde PostgreSQL y la envía como JSON; el frontend consume ese JSON mediante servicios, lo transforma a una estructura cómoda para React y lo renderiza en componentes visuales.

Ejemplo: módulo de ventas

```
PostgreSQL
  tabla ventas + usuario + metodo_pago + cliente + detalle_ventas_medicamento
  ↓
backend/src/models/ventasModel.js
  obtenerVentas()
  ↓
backend/src/controllers/ventasController.js
  listarVentas()
  ↓
backend/src/routes/ventasRoutes.js
  GET /api/ventas
  ↓
src/services/ventasService.js
  obtenerVentas()
  ↓
src/pages/Ventas.jsx
  useEffect(), setVentas(), filtros y paginación
  ↓
src/components/ventas/VentasTable.jsx
  tabla visible para el usuario
```

En `ventasModel.js`, la función `obtenerVentas()` ejecuta una consulta SQL que une ventas con usuario, método de pago, cliente y detalle de medicamentos. Después, el controlador `listarVentas()` responde con un objeto JSON:

```
{
  "data": [...ventas],
  "total": 8
}
```

En el frontend, `src/services/ventasService.js` llama a `fetch(`${API_URL}/ventas`)`, valida que la respuesta sea correcta y normaliza los campos. Por ejemplo, convierte `total_venta` a número, asigna `id` y formatea datos de fecha y cliente.

Finalmente, `src/pages/Ventas.jsx` guarda el resultado en estado con `setVentas`, calcula ventas filtradas y paginadas, y las pasa a `VentasTable` para mostrarlas en pantalla.

4. Dependencias: package.json Y package-lock.json

El proyecto utiliza un solo `package.json` y un solo `package-lock.json` en la raíz. Esto significa que frontend y backend comparten la misma instalación de dependencias en `node_modules/`.

Qué es package.json

`package.json` describe el proyecto Node.js: nombre, versión, tipo de módulos, scripts disponibles y dependencias requeridas para ejecutar o desarrollar la aplicación. En este proyecto también concentra los comandos de frontend, backend y base de datos.

Qué es package-lock.json

`package-lock.json` registra las versiones exactas instaladas de cada dependencia y subdependencia. Su objetivo es que otra persona pueda ejecutar `npm install` y obtener un entorno consistente.

Por qué hay uno solo para todo

Como el frontend y el backend están dentro del mismo repositorio y se ejecutan desde la raíz, basta con un solo manifiesto de Node. Esto simplifica la instalación para un proyecto académico: una sola carpeta, un solo `npm install`, un solo `node_modules` y scripts centralizados para levantar cada parte.

Dependencia	Parte del proyecto	Uso
<code>react</code> , <code>react-dom</code>	Frontend	Construcción de interfaces y renderizado de componentes.
<code>react-router-dom</code>	Frontend	Rutas internas como <code>/ventas</code> , <code>/clientes</code> y <code>/login</code> .
<code>recharts</code>	Frontend	Gráficas del dashboard y componentes visuales de métricas.
<code>express</code>	Backend	Servidor HTTP y API REST.
<code>cors</code>	Backend	Permite que el frontend Vite consuma la API Express desde otro puerto.
<code>dotenv</code>	Backend	Carga variables desde <code>.env.local</code> o <code>.env.example</code> .
<code>pg</code>	Backend	Conexión y consultas a PostgreSQL mediante un pool.
<code>vite</code> , <code>@vitejs/plugin-react</code>	Desarrollo frontend	Servidor de desarrollo, build y soporte para React.
<code>eslint</code>	Desarrollo	Revisión estática de código.

5. Lógica Del Proyecto Y Su Ubicación

La lógica del proyecto está separada por responsabilidad. El backend se encarga de la lógica de negocio y acceso a datos; el frontend se encarga de la presentación, interacción del usuario y consumo de la API.

Tipo de lógica	Ubicación	Responsabilidad
Lógica de negocio	<code>backend/src/controllers</code>	Coordina validaciones, llamadas a modelos y respuestas HTTP.
Acceso a datos	<code>backend/src/models</code>	Ejecuta consultas SQL, inserciones, actualizaciones y transacciones.
Validación backend	<code>backend/src/validators</code>	Comprueba que los datos recibidos tengan estructura y valores válidos.
Servicios frontend	<code>src/services</code>	Centraliza llamadas a la API y normaliza respuestas para React.
Presentación	<code>src/pages</code> y <code>src/components</code>	Muestra pantallas, formularios, tablas, filtros, modales y gráficas.
Permisos de interfaz	<code>src/utils/permisos.js</code>	Define módulos visibles y rutas permitidas según rol.

Un ejemplo claro es la venta: `VentaForm` construye los datos de la venta, `ventasService.crearVenta` envía la petición, `ventasController.crearVenta` valida y coordina, y `ventasModel.crearVenta` registra la venta y descuenta stock dentro de una transacción SQL.

6. Validaciones

El proyecto valida datos en dos niveles: frontend y backend. Esta doble validación es importante porque el frontend mejora la experiencia del usuario, pero el backend es la capa que realmente protege la base de datos.

Validaciones en frontend

En `src/utils/validaciones.js` se concentran funciones reutilizables para limpiar y validar campos. Hay límites de longitud, validación de nombres, teléfonos, correos, direcciones, lotes, stock, precios y contraseñas.

En formularios como `src/components/ventas/VentaForm.jsx`, la validación evita enviar una venta sin método de pago, sin medicamentos, con cantidades no numéricas o superiores al stock disponible del lote.

```
if (!formulario.id_metPag) {
  nuevosErrores.id_metPag = 'El metodo de pago es requerido.'
}

if (formulario.detalles.length === 0) {
  nuevosErrores.detalles = 'Debe agregar al menos un medicamento.'
}
```

Validaciones en backend

En `backend/src/validators` se validan los datos que llegan a la API. Por ejemplo, `ventasValidator.js` revisa que el usuario y el método de pago sean numéricos y positivos, que exista al menos un detalle y que cada detalle tenga medicamento, lote, cantidad y precio válidos.

La validación de login vive en `authValidator.js`, donde se comprueba que el usuario no esté vacío, tenga formato permitido y que la contraseña exista y no exceda el máximo definido.

Manejo de errores

Los controladores usan `try/catch` y envían errores a `next(error)`. Al final, `backend/src/middleware/errorHandler.js` registra el error en consola y responde con un JSON de error interno.

```
app.use('/api', apiRoutes)
app.use(errorHandler)
```

Cuando el error es una validación conocida, el controlador responde con estado `400`. Cuando la venta no existe, responde `404`. Los errores inesperados pasan al middleware general y se responden como `500`.

7. Seguridad Del Proyecto

La seguridad del proyecto se basa en autenticación, manejo de sesión en frontend, protección de rutas por rol, validaciones y consultas parametrizadas hacia PostgreSQL.

Mecanismo	Ubicación	Explicación
Login	<code>backend/src/controllers/authController.js</code>	Recibe usuario y contraseña, valida formato y busca al usuario en base de datos.
Hash de contraseña	<code>backend/src/utlis/password.js</code>	Usa PBKDF2 con salt, 100000 iteraciones, SHA-512 y comparación segura con <code>timingSafeEqual</code> .
Sesión frontend	<code>src/services/authService.js</code>	Guarda la sesión en <code>localStorage</code> con la clave <code>farmacia_inclusiva_usuario</code> .
Protección de rutas	<code>src/components/common/ProtectedRoute.jsx</code>	Si no hay sesión redirige a <code>/login</code> ; si el rol no tiene permiso redirige a su ruta inicial.
Permisos por rol	<code>src/utlis/permisos.js</code>	Admin accede a todos los módulos principales; cajero accede a Inicio, Medicamentos, Clientes y Ventas.
Consultas parametrizadas	<code>backend/src/models</code>	Las consultas usan parámetros como <code>\$1</code> , <code>\$2</code> , reduciendo riesgo de inyección SQL.
CORS	<code>backend/src/app.js</code>	Permite que el frontend consuma la API desde el navegador durante desarrollo.

Observación técnica: la protección por rol está implementada principalmente en el frontend. El backend valida datos y contraseñas, pero no se observa un middleware global de autorización por token o sesión para proteger cada endpoint de la API. Para una versión productiva, sería recomendable agregar autenticación de API con tokens o sesiones de servidor y autorización por rol también en Express.

8. Mapa De Carpetas Y Archivos

```
Proyecto-Programacion-Web/
├── package.json
│   Scripts y dependencias compartidas por frontend y backend.
├── package-lock.json
│   Versiones exactas instaladas por npm.
├── vite.config.js
│   Configuración de Vite para React.
├── docker-compose.yml
│   Servicio PostgreSQL 16, puerto local 5433 y carga de scripts SQL.
├── .env.example
│   Variables de ejemplo para API y conexión PostgreSQL.
├── index.html
│   HTML base donde Vite monta la aplicación React.
├── public/
│   Recursos estáticos públicos como favicon e iconos.
├── src/
│   Frontend React.
│   ├── main.jsx
│   │   Punto de entrada del frontend.
│   ├── App.jsx
│   │   Define rutas y protección general.
│   ├── App.css e index.css
│   │   Estilos globales de la aplicación.
│   ├── pages/
│   │   Pantallas principales: Login, Inicio, Dashboard, Ventas, Clientes, etc.
│   ├── components/
│   │   Componentes reutilizables: tablas, formularios, filtros, modales y gráficos.
│   ├── services/
│   │   Funciones que consumen la API: ventasService, authService, clientesService, etc.
│   ├── context/
│   │   Contexto de autenticación.
│   ├── hooks/
│   │   Hooks propios como useAuth.
│   ├── utils/
│   │   Validaciones, sanitización y permisos por rol.
│   └── data/
│       Datos locales auxiliares o de respaldo.
├── backend/
│   └── src/
│       Backend Express.
│       ├── server.js
│       │   Arranque del servidor.
│       ├── app.js
│       │   Configuración de Express.
│       ├── config/
│       │   Variables de entorno y conexión PostgreSQL.
│       ├── routes/
│       │   Endpoints agrupados por módulo.
│       ├── controllers/
│       │   Lógica de petición y respuesta.
│       ├── models/
│       │   Consultas SQL y transacciones.
│       ├── validators/
│       │   Validaciones backend.
```

```
| | | middleware/  
| | |   Middleware de errores.  
| | | | | utils/  
| | | | |   Utilidades como hash de contraseñas.  
| | BaseDeDatos/  
| | | farmacia_inclusiva.sql  
| | |   Schema principal de PostgreSQL.  
| | | | | migrations/  
| | | | |   Cambios incrementales de base de datos.  
| | | | | seeds/  
| | | | |   Datos iniciales para proveedores y catálogo.  
| | docs/  
| |   Documentación del proyecto.
```

9. Flujo De Una Petición Completa: Registrar Una Venta

Este flujo describe qué ocurre cuando el usuario registra una venta desde la interfaz hasta que los datos quedan guardados en PostgreSQL.

1. El usuario entra al módulo `/ventas`. La ruta está protegida por `ProtectedRoute`, que revisa si existe sesión y si el rol tiene permiso.
2. En `Ventas.jsx`, el usuario presiona el botón para abrir el modal de registro. El modal muestra `VentaForm`.
3. `VentaForm` carga opciones con `Promise.all`: métodos de pago, clientes y medicamentos disponibles. Estas opciones vienen de `ventasService` y de endpoints como `/ventas/opciones/medicamentos`.
4. El usuario selecciona cliente opcional, método de pago, medicamento, lote y cantidad. El formulario valida que el lote exista, que la cantidad sea positiva y que no supere el stock disponible.
5. Al enviar el formulario, `VentaForm` construye un objeto como este:

```
{
  "id_usuario": 1,
  "id_metPag": 2,
  "id_cliente": null,
  "detalles": [
    {
      "id_medicamento": 5,
      "id_lote": 12,
      "cantidad": 2,
      "precio_unitario": 75.50
    }
  ]
}
```

6. `Ventas.jsx` ejecuta `crearVenta(nuevaVenta)`, definido en `src/services/ventasService.js`.
7. El servicio envía una petición `POST` a `/api/ventas` con JSON.
8. En el backend, `backend/src/routes/ventasRoutes.js` dirige la petición hacia `ventasController.crearVenta`.
9. El controlador llama a `validarVenta`. Si los datos son incorrectos, responde `400` con los detalles del error.
10. Si los datos son válidos, el controlador llama a `ventasModel.crearVenta`.
11. El modelo abre una transacción SQL con `BEGIN`, inserta la venta, valida el lote disponible con `FOR UPDATE`, inserta el detalle, descuenta stock y actualiza el total de la venta.
12. Si todo sale bien, ejecuta `COMMIT`. Si ocurre un error, ejecuta `ROLLBACK` para no dejar datos incompletos.
13. El backend responde con estado `201` y un JSON con el ID de la venta y el total.
14. El frontend cierra el modal, muestra el mensaje de éxito y vuelve a llamar `obtenerVentas()` para refrescar la tabla.

Click en "Registrar venta"

- > VentaForm valida datos
- > ventasService.crearVenta()
- > POST /api/ventas
- > ventasRoutes
- > ventasController.crearVenta()
- > validarVenta()
- > ventasModel.crearVenta()
- > BEGIN / INSERT / UPDATE stock / COMMIT
- > JSON 201
- > setVentas(ventasActualizadas)
- > pantalla actualizada

10. Preguntas Y Respuestas Frecuentes Sobre El Proyecto

¿Dónde está el backend?

El backend está en `backend/src/`. Ahí se encuentran `server.js`, `app.js`, rutas, controladores, modelos, validadores, configuración y middleware. Se levanta con `npm run dev:api`.

¿Cómo se muestran los datos en pantalla?

Los datos salen de PostgreSQL, pasan por un modelo backend, un controlador y una ruta Express. Luego un servicio en `src/services` consume la API y una página React guarda los datos en estado para renderizarlos en tablas, formularios o gráficos.

¿Dónde están las dependencias?

Están declaradas en el `package.json` de la raíz del proyecto. Las versiones exactas instaladas se registran en `package-lock.json`.

¿Por qué hay un solo package.json para todo?

Porque frontend y backend forman parte del mismo proyecto Node y se administran desde la raíz. Esto permite instalar todo con un solo `npm install` y ejecutar scripts centralizados como `npm run dev`, `npm run dev:api` y `npm run build`.

¿Dónde está la lógica del proyecto?

La lógica de negocio está principalmente en `backend/src/controllers` y `backend/src/models`. La lógica de presentación está en `src/pages` y `src/components`. La conexión entre ambas partes está en `src/services`.

¿Cómo funcionan las validaciones?

El frontend valida formularios para evitar errores antes de enviar datos. El backend vuelve a validar en `backend/src/validators` antes de guardar o consultar información crítica. Los errores controlados se responden con estados HTTP como `400` o `404`.

¿Qué seguridad usa el proyecto?

Usa autenticación por usuario y contraseña, hash de contraseña con PBKDF2, sesión guardada en `localStorage`, rutas protegidas en React y permisos por rol admin/cajero. Además, los modelos usan consultas parametrizadas hacia PostgreSQL.

¿En qué carpeta está cada archivo importante?

El frontend está en `src/`, el backend en `backend/src/`, la base de datos en `BaseDeDatos/`, la configuración de Docker en `docker-compose.yml` y las dependencias en `package.json`.

Anexo: Comandos Principales

Comando	Uso
<code>npm run dev</code>	Inicia el frontend con Vite.
<code>npm run dev:api</code>	Inicia el backend Express.
<code>npm run db:up</code>	Levanta PostgreSQL con Docker Compose.
<code>npm run db:down</code>	Detiene los contenedores.
<code>npm run db:reset</code>	Reinicia la base de datos eliminando el volumen y cargando scripts iniciales.
<code>npm run build</code>	Compila el frontend para producción.
<code>npm run lint</code>	Ejecuta ESLint sobre el proyecto.

Documento generado para el proyecto Farmacia Inclusiva. La explicación se basa en la estructura y archivos presentes en el repositorio local.